

Aim: Pre-Processing and Classification in Data Mining (Python Jupiter Lab).

1.Importing Basic libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

2.Importing visualization libraries.

```
import micropip
await micropip.install(['seaborn'])
```

3.Using Preprocessing and modeling with sklearn.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler,LabelEncoder
from sklearn.impute import SimpleImputer
```

4.Importing Classification models for biased result.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

5.Importing the dataset by using the url.

```
from pyodide.http import open_url
import pandas as pd
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
with open_url(url) as f:
    df = pd.read_csv(f)
```

df.head()

Saved to this PC

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

6.Getting the information from the dataset.

```
print("Dataset shape:",df.shape)
print(df.info())
```

```
Dataset shape: (891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 66.2+ KB
None
```

7.Describing the dataset.

```
print(df.describe())
```

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

8.Finding the missing values per column.

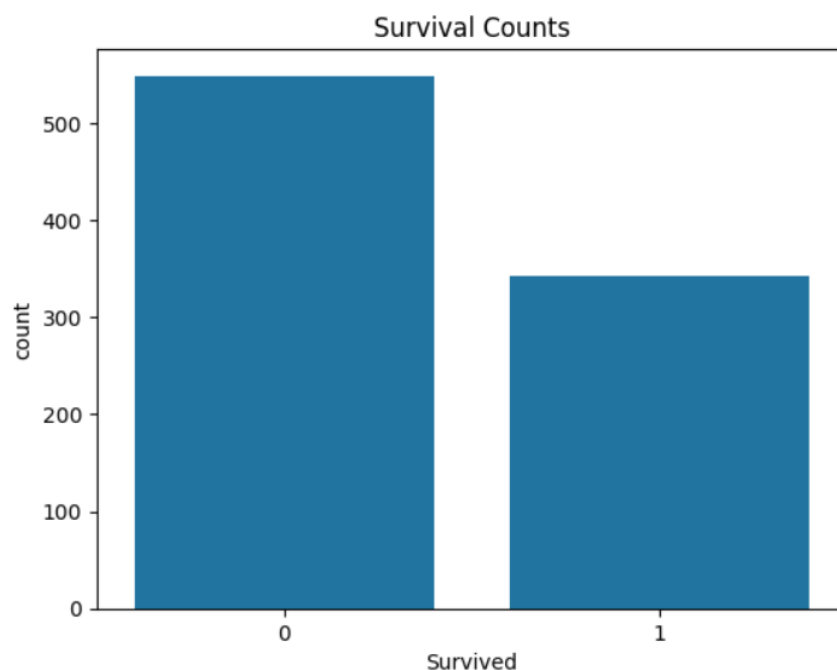
```
print("Missing Values per column:")  
print(df.isnull().sum())
```

```
Missing Values per column:  
PassengerId      0  
Survived          0  
Pclass           0  
Name             0  
Sex              0  
Age             177  
SibSp            0  
Parch            0  
Ticket           0  
Fare             0  
Cabin           687  
Embarked         2  
dtype: int64
```

7. Visualizing the survival count of male and female.

```
sns.countplot(x='Survived', data=df)  
plt.title("Survival Counts")
```

```
Text(0.5, 1.0, 'Survival Counts')
```



8. Handling the missing values with the help of SimpleImputer, and strategy using median and most_frequent, transforming the data with embarked, and dropping the column that has too many missing values using Cabin.

```
imputer=SimpleImputer(strategy='median')
df['Age']=imputer.fit_transform(df[['Age']])

imputer_mode=SimpleImputer(strategy='most_frequent')
df[['Embarked']]=imputer_mode.fit_transform(df[['Embarked']])

if 'Cabin' in df.columns:
    df.drop('Cabin',axis=1,inplace=True)

print(df.isnull().sum())
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            0
SibSp           0
Parch           0
Ticket          0
Fare            0
Embarked        0
dtype: int64
```

1.Using encoder to convert characters into numbers.

```
label_enc = LabelEncoder()
df['Sex'] = label_enc.fit_transform(df['Sex'])
df['Embarked'] = label_enc.fit_transform(df['Embarked'])

df[['Sex', 'Embarked']].head()
```

OUTPUT:

	Sex	Embarked
0	1	2
1	0	0
2	0	2
3	0	2
4	1	2

2.Feature selection and scaling, where converting scaled data back to dataframe for easy plotting. Plotting boxplot of scaled feature.

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from pyodide.http import open_url

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

with open_url(url) as f:
    X = pd.read_csv(f)

features = ('Age', 'Fare', 'Sex', 'Embarked')

if all(col in X.columns for col in features):
    X['Embarked'] = X['Embarked'].map({'C':0, 'Q':1, 'S':2})
    X['Sex'] = X['Sex'].map({'male':0, 'female':1})

    X = X.dropna(subset = features)

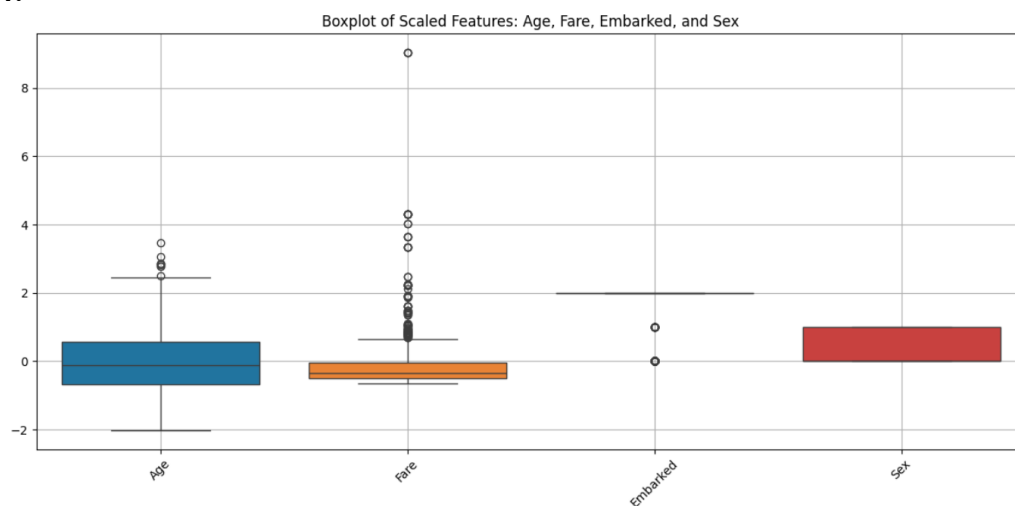
    scalar=StandardScaler()
    X_scaled=scalar.fit_transform(X[['Age', 'Fare']])
    X_scaled_df = pd.DataFrame(X_scaled, columns = ['Age', 'Fare'])

    X_scaled_df['Embarked'] = X['Embarked']
    X_scaled_df['Sex'] = X['Sex']

    plt.figure(figsize = (12,6))
    sns.boxplot(data = X_scaled_df)
    plt.title("Boxplot of Scaled Features: Age, Fare, Embarked, and Sex")
    plt.xticks(rotation = 45)
    plt.grid(True)
    plt.tight_layout()
    plt.show()

else:
    print(f"One or more of the specified features {features} are not found in your dataset")

```

OUTPUT:**3.Splitting dataset (Train-test split).**

```
from sklearn.model_selection import train_test_split

y = X['Survived']

X = X.drop(columns=['Survived'])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("Training set size:", X_train.shape)
print("Training set size:", X_test.shape)
```

OUTPUT:

```
Training set size: (623, 10)
Training set size: (268, 10)
```